Variante GitOps : déployer sur EKS avec Argo CD

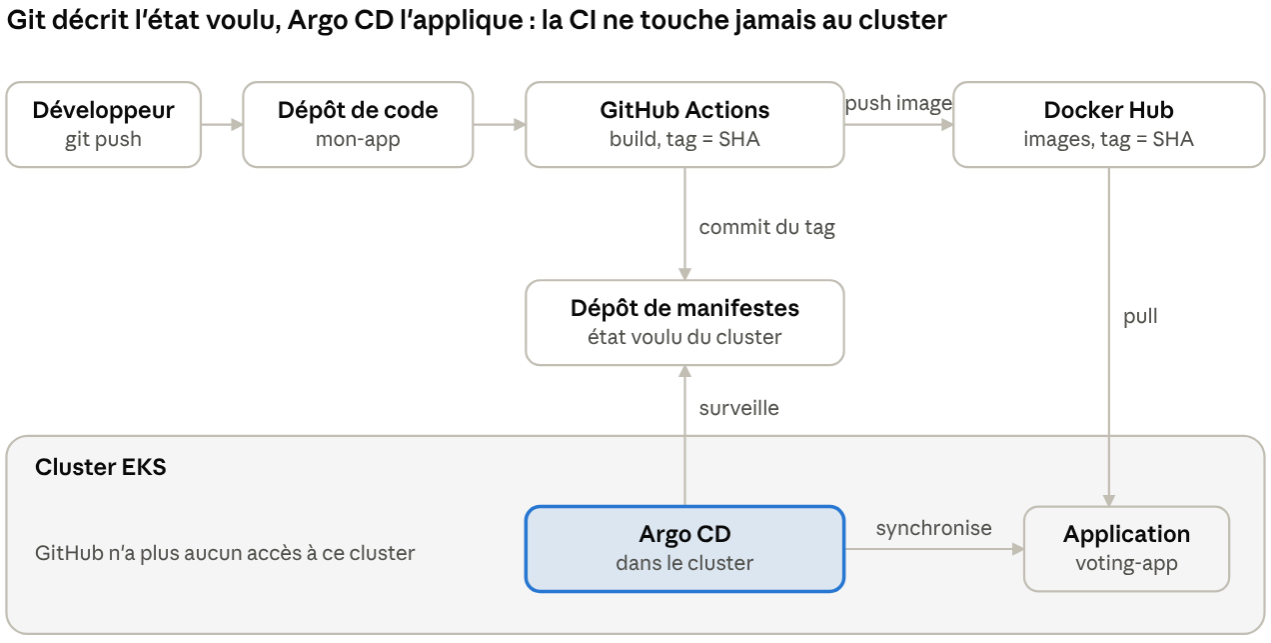
Oct 27, 2026 · Dally NTOLO

Principe et différences avec le projet 2

Dans le projet 2, GitHub Actions construit les images puis se connecte au cluster pour déployer. Ici, la CI ne touche plus jamais au cluster : elle construit l'image et écrit le nouveau tag dans un dépôt Git. Argo CD, installé dans le cluster, voit le changement et met le cluster à jour. C'est le principe du GitOps : Git est la seule source de vérité.

Enchevêtrement après un `git push` :

1. GitHub Actions construit les images et les publie avec le SHA du commit comme tag.
2. Le pipeline modifie le tag dans le dépôt de manifestes et pousse le changement.
3. Argo CD détecte le commit et synchronise le cluster.
4. Si quelqu'un modifie le cluster à la main, Argo CD annule la modification.



déploiement GitOps avec Argo CD

La CI s'arrête à Git : c'est Argo CD, dans le cluster, qui va chercher le changement et l'applique.

	Projet 2 (kubectl dans la CI)	Cette variante (Argo CD)
Qui déploie	GitHub Actions	Argo CD, dans le cluster
Accès de GitHub au cluster	Rôle IAM OIDC et access entry	Aucun
Dépôts Git	Un seul	Deux : code et manifestes
Retour arrière	Relancer un ancien déploiement	`git revert`
Modification manuelle	Reste en place	Annulée automatiquement

Cette variante n'a pas été réalisée dans le projet 2 : c'est une alternative documentée, non testée sur le cluster.

Prérequis et variables

Ce guide part de zéro : il crée le cluster, le chiffrement des Secrets, la porte d'entrée en HTTPS, puis met en place le déploiement par Argo CD. Rien n'est à préparer à l'avance, hors un compte AWS, un domaine géré dans Route 53, un compte GitHub et un compte Docker Hub.

Outils à installer : AWS CLI v2 (configuré avec `aws configure`), eksctl, kubectl, helm, git, docker, GitHub CLI (`gh`), et optionnellement la ligne de commande `argocd`.

```
export AWS_REGION=us-east-1
export CLUSTER=mon-projet
export NS=mon-app
export DOMAIN=app.exemple.com
export GH_USER=mon-compte-github
export APP_REPO=mon-app                 # code, Dockerfiles, workflow
export MANIFESTS_REPO=mon-app-manifests # manifestes Kubernetes
export DH_USER=mon-compte-dockerhub
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output
text)
```

Étape 1 : créer le cluster EKS

Compter 15 à 20 minutes.

```
cat > cluster.yaml <<EOF
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: ${CLUSTER}
  region: ${AWS_REGION}
iam:
  withOIDC: true
managedNodeGroups:
  - name: ng-1
    instanceType: t3.medium
    desiredCapacity: 2
    minSize: 2
    maxSize: 3
EOF

eksctl create cluster -f cluster.yaml
kubectl get nodes
```

Volumes persistants (nécessaires à la base de données) : pilote EBS et StorageClass gp3.

```
eksctl create iamserviceaccount --name ebs-csi-controller-sa --namespace
kube-system \
  --cluster $CLUSTER --role-name ${CLUSTER}-ebs-csi --role-only \
  --attach-policy-arn arn:aws:iam::aws:policy/service-
role/AmazonEBSCSIDriverPolicy --approve

eksctl create addon --name aws-ebs-csi-driver --cluster $CLUSTER \
  --service-account-role-arn arn:aws:iam::${ACCOUNT_ID}:role/${CLUSTER}-ebs-
csi --force

kubectl apply -f - <<EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
volumeBindingMode: WaitForFirstConsumer
EOF
```

Critère de réussite : les 2 nœuds sont `Ready`.

Étape 2 : chiffrer les Secrets avec une clé KMS

```
KMS_ARN=$(aws kms create-key --description "${CLUSTER} secrets" \
  --query KeyMetadata.Arn --output text)
aws kms create-alias --alias-name alias/${CLUSTER}-secrets --target-key-id
$KMS_ARN

eksctl utils enable-secrets-encryption --cluster $CLUSTER \
  --key-arn $KMS_ARN --region $AWS_REGION
```

L'opération prend quelques minutes et ne coupe pas le cluster.

Étape 3 : installer la porte d'entrée (Gateway API et NGINX Gateway Fabric)

La Gateway API est le successeur officiel de l'Ingress. On installe ici le contrôleur ; la Gateway elle-même et les routes seront décrites dans Git à l'étape 5. Vérifier dans la documentation de NGINX Gateway Fabric la version de la Gateway API compatible.

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.2.1/standard-install.yaml

helm install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric \
  --create-namespace -n nginx-gateway

kubectl get gatewayclass          # doit afficher la classe nginx
```

Étape 4 : installer cert-manager (certificats HTTPS)

```
helm repo add jetstack https://charts.jetstack.io && helm repo update
helm install cert-manager jetstack/cert-manager -n cert-manager \
  --create-namespace --set crds.enabled=true
kubectl -n cert-manager get pods
```

L'émetteur Let's Encrypt et le certificat seront eux aussi décrits dans Git (étape 5).

Étape 5 : organiser les deux dépôts Git

On sépare le code (qui change souvent) des manifestes (qui décrivent l'état voulu du cluster). Le dépôt de manifestes est celui qu'Argo CD surveille.

```
gh repo create $GH_USER/$MANIFESTS_REPO --private --clone
cd $MANIFESTS_REPO
mkdir -p base overlays/prod
```

Structure du dépôt de manifestes :

```
mon-app-manifests/
  base/
    kustomization.yaml
    vote.yaml  result.yaml  worker.yaml  redis.yaml  postgres.yaml
    gateway.yaml  secrets-config.yaml
  overlays/prod/
    kustomization.yaml
```

Placer dans `base/` les manifestes de l'application, décrits ci-dessous (Deployments, Services, base de données, Gateway, routes, certificat). Les Secrets ne vont jamais dans Git : les créer à la main une fois dans le cluster, ou utiliser un outil dédié comme External Secrets.

Un service applicatif (`base/vote.yaml`). Les fichiers `result.yaml` et `redis.yaml` suivent le même modèle ; `worker.yaml` aussi, sans Service. Les ports et noms de variables dépendent des images utilisées :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vote
spec:
  replicas: 2
  selector:
    matchLabels: {app: vote}
  template:
    metadata:
      labels: {app: vote}
    spec:
      containers:
        - name: vote
          image: mon-compte-dockerhub/vote:initial
          ports:
            - containerPort: 80
          resources:
            requests: {cpu: 100m, memory: 128Mi}
---
apiVersion: v1
kind: Service
```

```
metadata:
  name: vote
spec:
  selector: {app: vote}
  ports:
    - port: 80
      targetPort: 80
```

Les services se joignent par leur nom (`redis` , `db`), sans adresse IP en dur.

La base de données (`base/postgres.yaml`), avec un volume gp3 :

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db
  replicas: 1
  selector:
    matchLabels: {app: db}
  template:
    metadata:
      labels: {app: db}
    spec:
      containers:
        - name: postgres
          image: postgres:16
          envFrom:
            - secretRef: {name: db-credentials}
          ports:
            - containerPort: 5432
          volumeMounts:
            - {name: data, mountPath: /var/lib/postgresql/data, subPath:
pgdata}
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        storageClassName: gp3
        resources:
          requests: {storage: 5Gi}
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: db
spec:
  selector: {app: db}
  ports:
    - port: 5432
```

La porte d'entrée, le certificat et la redirection HTTPS (`base/gateway.yaml`). Remplacer le domaine et l'adresse e-mail :

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: gw
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      port: 80
      protocol: HTTP
    - name: https
      port: 443
      protocol: HTTPS
      hostname: app.exemple.com
      tls:
        mode: Terminate
        certificateRefs:
          - name: app-tls
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: routes
spec:
  parentRefs:
    - {name: gw, sectionName: https}
  hostnames: ["app.exemple.com"]
  rules:
    - matches: [{path: {type: PathPrefix, value: /vote}}]
      filters:
        - type: URLRewrite
          urlRewrite:
            path: {type: ReplacePrefixMatch, replacePrefixMatch: /}
      backendRefs: [{name: vote, port: 80}]
    - matches: [{path: {type: PathPrefix, value: /result}}]
      filters:
        - type: URLRewrite
```

```
          urlRewrite:
            path: {type: ReplacePrefixMatch, replacePrefixMatch: /}
      backendRefs: [{name: result, port: 80}]
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: http-redirect
spec:
  parentRefs:
    - {name: gw, sectionName: http}
  rules:
    - filters:
        - type: RequestRedirect
          requestRedirect: {scheme: https, statusCode: 301}
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: votre.email@exemple.com
    privateKeySecretRef: {name: letsencrypt-key}
    solvers:
      - http01:
          gatewayHTTPRoute:
            parentRefs:
              - {name: gw, namespace: mon-app, kind: Gateway, sectionName:
http}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: app-tls
spec:
  secretName: app-tls
  dnsNames: ["app.exemple.com"]
  issuerRef: {name: letsencrypt, kind: ClusterIssuer}
```

Si l'application gère déjà les préfixes `/vote` et `/result`, retirer les filtres `URLRewrite`.

```
# base/kustomization.yaml
resources:
  - vote.yaml
  - result.yaml
  - worker.yaml
  - redis.yaml
  - postgres.yaml
  - gateway.yaml
```

```
# overlays/prod/kustomization.yaml
namespace: mon-app
resources:
  - ../../base
images:
  - name: mon-compte-dockerhub/vote
    newTag: initial
  - name: mon-compte-dockerhub/result
    newTag: initial
  - name: mon-compte-dockerhub/worker
    newTag: initial
```

```
kubectl kustomize overlays/prod | head -30     # vérifier le rendu
git add . && git commit -m "Initial manifests" && git push
```

Le champ `images` est la seule chose que la CI modifiera : le tag de chaque image.

Étape 6 : installer Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side \
  -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
```

Accès à l'interface (sans l'exposer sur Internet) :

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

Ouvrir https://localhost:8080 (le navigateur signale un certificat non reconnu, c'est normal en local), identifiant `admin` et le mot de passe affiché ci-dessus. Le mot de passe initial ne se copie pas dans un message ni dans un document.

Changer ensuite le mot de passe dans l'interface (User Info), puis supprimer le secret initial :

```
kubectl -n argocd delete secret argocd-initial-admin-secret
```

Donner à Argo CD l'accès en lecture au dépôt privé de manifestes. Créer sur GitHub un jeton à accès limité à ce seul dépôt et en lecture seule, le mettre dans une variable d'environnement, puis :

```
argocd login localhost:8080 --username admin --insecure
argocd repo add https://github.com/$GH_USER/$MANIFESTS_REPO.git \
  --username $GH_USER --password "$ARGOCD_REPO_TOKEN"
argocd repo list
```

Critère de réussite : le dépôt apparaît avec l'état `Successful`.

Étape 7 : déclarer l'application dans Argo CD

Une ressource `Application` indique à Argo CD quoi surveiller et où déployer (`application.yaml`) :

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: mon-app
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/mon-compte-github/mon-app-manifests.git
    targetRevision: main
    path: overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: mon-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

- `automated` : Argo CD synchronise sans attendre un clic.

- `prune: true` : une ressource retirée de Git est supprimée du cluster.
- `selfHeal: true` : une modification manuelle du cluster est annulée.
- Le `finalizer` fait supprimer les ressources de l'application quand on supprime l'`Application`.

Avant la première synchronisation, créer le namespace et le Secret de la base (ils ne sont pas dans Git) :

```
kubectl create namespace $NS
kubectl -n $NS create secret generic db-credentials \
  --from-literal=POSTGRES_USER=app \
  --from-literal=POSTGRES_PASSWORD='<choisir-un-mot-de-passe>'
```

Puis déclarer l'application :

```
kubectl apply -f application.yaml
kubectl -n argocd get applications
kubectl -n $NS get pods
```

Vérifier dans l'interface que l'application est `Synced` et `Healthy`.

Étape 8 : adapter le pipeline GitHub Actions

Le workflow ne se connecte plus à AWS ni au cluster : plus de rôle OIDC, plus de `kubectl`. Il construit, publie, puis écrit le nouveau tag dans le dépôt de manifestes.

Secrets à créer dans le dépôt de code : `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`, et `MANIFESTS_TOKEN` (jeton GitHub limité au dépôt de manifestes, en écriture).

```
gh secret set DOCKERHUB_USERNAME --repo $GH_USER/$APP_REPO
gh secret set DOCKERHUB_TOKEN    --repo $GH_USER/$APP_REPO
gh secret set MANIFESTS_TOKEN    --repo $GH_USER/$APP_REPO
```

`.github/workflows/ci.yml` :

```
name: ci
on:
  push:
    branches: [main]
jobs:
  build-and-bump:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - uses: docker/build-push-action@v6
        with:
          context: ./vote
          push: true
          tags: mon-compte-dockerhub/vote:${{ github.sha }}
      # Répéter pour result et worker

      - uses: actions/checkout@v4
        with:
          repository: mon-compte-github/mon-app-manifests
          token: ${{ secrets.MANIFESTS_TOKEN }}
          path: manifests

      - name: Mettre à jour les tags dans le dépôt de manifestes
        run: |
          cd manifests/overlays/prod
          kustomize edit set image mon-compte-dockerhub/vote=mon-compte-
dockerhub/vote:${GITHUB_SHA}
          kustomize edit set image mon-compte-dockerhub/result=mon-compte-
dockerhub/result:${GITHUB_SHA}
          kustomize edit set image mon-compte-dockerhub/worker=mon-compte-
dockerhub/worker:${GITHUB_SHA}
          git config user.name "ci-bot"
          git config user.email "ci-bot@users.noreply.github.com"
          git commit -am "Deploy ${GITHUB_SHA}"
          git push
```

`kustomize` est généralement préinstallé sur les machines GitHub ; sinon, l'installer dans une étape précédente. Si seul un service a changé, ne reconstruire que celui-là.

Étape 9 : DNS puis vérifier, tester le retour arrière et la sécurité

1. Pointer le domaine vers le load balancer. Après la synchronisation, Argo CD a créé la Gateway et AWS a fabriqué un load balancer. Créer l'enregistrement CNAME dans Route 53 :

```
kubectl get svc -A | grep LoadBalancer
LB=$(kubectl get svc -A -o jsonpath='{.items[?
(@.spec.type=="LoadBalancer")].status.loadBalancer.ingress[0].hostname}')
ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name <domaine-racine> \
  --query 'HostedZones[0].Id' --output text)

aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-
batch "{
  \"Changes\": [{\"Action\": \"UPSERT\", \"ResourceRecordSet\": {
    \"Name\": \"${DOMAIN}\", \"Type\": \"CNAME\", \"TTL\": 60,
    \"ResourceRecords\": [{\"Value\": \"${LB}\"}]}}]}"
```

2. Attendre le certificat. Let's Encrypt valide le domaine dès que le DNS est propagé (quelques minutes) :

```
kubectl -n $NS get certificate -w     # attendre READY = True
curl -I http://$DOMAIN                # 301 vers HTTPS
curl -I https://$DOMAIN/vote          # 200
```

3. Vérifier le déploiement automatique.

Test de bout en bout : modifier le code, faire un `git push` sur `main`, puis suivre :

```
gh run watch --repo $GH_USER/$APP_REPO
argocd app get mon-app            # Synced et Healthy, avec le nouveau tag
kubectl -n $NS get deploy vote -o
jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Par défaut Argo CD vérifie Git environ toutes les 3 minutes. Pour forcer : `argocd app sync mon-app`.

Test de l'auto-réparation : modifier le cluster à la main et constater qu'Argo CD annule le changement.

```
kubectl -n $NS scale deploy vote --replicas=5
kubectl -n $NS get deploy vote -w     # revient au nombre déclaré dans Git
```

Retour arrière : annuler le dernier déploiement revient à annuler un commit.

```
cd $MANIFESTS_REPO && git revert HEAD --no-edit && git push
```

Points de sécurité :

- GitHub n'a plus aucun accès au cluster : le rôle IAM `...-github-deploy` et son access entry ne servent plus et peuvent être supprimés.
- Les jetons GitHub sont limités à un seul dépôt et ont une date d'expiration.

- L'interface d'Argo CD reste en `port-forward`, ou derrière une authentification si on l'expose.
- Les Secrets ne sont jamais dans Git.

Étape 10 : nettoyage

Supprimer d'abord l'`Application` : grâce au finalizer, Argo CD retire lui-même les ressources déployées, dont la Gateway et donc le load balancer AWS. Attendre qu'elle ait disparu avant de continuer.

```
kubectl -n argocd delete application mon-app
kubectl -n argocd get applications          # doit être vide
kubectl -n $NS get pvc                      # supprimer les PVC restants

kubectl delete -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
kubectl delete namespace argocd
```

Puis, dans cet ordre, le reste de ce que le guide a créé :

```
# 1. DNS : supprimer l'enregistrement CNAME du domaine (console Route 53, ne
pas toucher à la zone)

# 2. Composants installés avec Helm
helm uninstall ngf -n nginx-gateway
helm uninstall cert-manager -n cert-manager
kubectl delete namespace nginx-gateway cert-manager $NS
kubectl delete -f https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.2.1/standard-install.yaml

# 3. Vérifier qu'il ne reste rien qui coûte
kubectl get svc -A | grep LoadBalancer     # doit être vide
kubectl get pv                              # doit être vide

# 4. Cluster
eksctl delete cluster --name $CLUSTER --region $AWS_REGION --wait

# 5. Clé KMS (suppression différée, 7 jours minimum)
aws kms delete-alias --alias-name alias/${CLUSTER}-secrets
aws kms schedule-key-deletion --key-id $KMS_ARN --pending-window-in-days 7
```

Enfin, sur GitHub, révoquer les jetons `MANIFESTS_TOKEN` et `ARGOCD_REPO_TOKEN`.